

Pair RDF and Cumulative Coordination Specification

Shared-Nighbor Structural Analysis for AtomisticFrameCollection

mdstats

2026-07-11

1. Purpose

This document specifies the pair radial-distribution function (RDF), cumulative coordination curve, and RDF feature-detection API in

`mdstats/analysis/rdf.py`

The module accepts an `AtomisticFrameCollection` with trajectory or ensemble semantics, including a single static frame. Each frame is analyzed independently. Velocities and physical time ordering are not required.

The revised implementation delegates all periodic pair geometry to

`mdstats/analysis/_neighbors.py`

so RDF, integer coordination, and bond-angle analysis use identical minimum-image and cutoff conventions.

2. Scope and motives

The RDF answers a pair-correlation question:

Relative to a selected center atom of group A , how does the probability of finding a selected atom of group B vary with radial distance?

The same raw pair distances also produce the mean cumulative coordination curve. RDF smoothing and feature detection are diagnostic post-processing; they never replace the authoritative raw pair-count estimator.

The RDF module remains responsible for:

- spherical-shell binning;
- framewise density normalization;
- identical- and distinct-selection normalization;
- cumulative coordination;
- Gaussian smoothing;
- first-peak and first-minimum detection;
- conversion of a detected minimum into `PairCutoff` provenance.

The shared neighbor layer remains responsible for:

- frame and cell validation;
- minimum-image displacement vectors;
- pair distances;
- self-pair exclusion;
- deterministic pair-counting semantics;
- globally safe radial-range validation.

3. Mathematical conventions

3.1 Partial RDF

For center group A and neighbor group B , the partial RDF is estimated from shell counts. For frame t , shell b spans

$$[r_b, r_{b+1}),$$

with exact shell volume

$$\Delta V_b = \frac{4\pi}{3}(r_{b+1}^3 - r_b^3).$$

For disjoint selections, let N_A and N_B be the selected atom counts and V_t the frame volume. The frame RDF estimator is

$$g_{AB}^{(t)}(r_b) = \frac{V_t n_b^{(t)}}{N_A N_B \Delta V_b},$$

where $n_b^{(t)}$ is the number of directed $A \rightarrow B$ pairs in the shell.

For identical selections $A = B$ with $N_A = N$, unordered physical pairs are counted once. The normalization is

$$g_{AA}^{(t)}(r_b) = \frac{2V_t n_b^{(t)}}{N(N-1) \Delta V_b}.$$

The collection RDF is the arithmetic mean over selected frames:

$$g_{AB}(r_b) = \frac{1}{N_f} \sum_t g_{AB}^{(t)}(r_b).$$

Framework normalization is required for variable-cell trajectories and ensembles.

3.2 Cumulative coordination

For disjoint groups, the mean number of B neighbors around one A center within radius r is

$$N_{AB}(r) = \frac{1}{N_f N_A} \sum_t \sum_{i \in A} \sum_{j \in B} \Theta(r - r_{ij}^{(t)}).$$

The implementation derives this directly from cumulative pair counts. It does not integrate a smoothed RDF.

For identical groups, each unordered pair contributes two neighbors, one to each atom. The cumulative count is multiplied by two before division by N .

3.3 Pair geometry

The shared neighbor layer returns minimum-image distances satisfying

$$r_{ij} < r_{\max}.$$

`compute_pair_rdf()` passes:

- `PairCounting.DIRECTED` for disjoint selections;
- `PairCounting.UNORDERED_IDENTICAL` for identical selections.

Partially overlapping explicit selections are rejected.

4. Public data structures

4.1 RDFResult

`@dataclass(slots=True)`

`class RDFResult:`

`species_a: str`

`species_b: str`

`r: NDArray[np.float64]`

`g_r: NDArray[np.float64]`

`counts: NDArray[np.int64]`

`bin_edges: NDArray[np.float64]`

```

shell_volumes: NDArray[np.float64]

cn_r: NDArray[np.float64]
coordination_number: NDArray[np.float64]

atom_indices_a: NDArray[np.int64]
atom_indices_b: NDArray[np.int64]
frame_indices: NDArray[np.int64]

n_frames: int
n_bins: int
r_max: float
average_volume: float

metadata: dict[str, Any] = field(default_factory=dict)

```

counts is the total raw pair count over selected frames. g_r is the mean of frame-normalized RDF curves, not a normalization performed from the average volume alone.

coordination_number is authoritative and nondecreasing.

4.2 RDFFeature

```

@dataclass(slots=True)
class RDFFeature:
    kind: str
    radius: float
    index: int
    value: float
    prominence: float | None
    width: float | None
    confidence: str
    smoothing_sigma: float
    stability_std: float | None
    metadata: dict[str, Any] = field(default_factory=dict)

```

Standard kind values produced by the module are "peak", "minimum", and "manual_cutoff". Standard confidence values are "high", "medium", and "low". Feature values are reported in physical units. Ambiguous first minima raise an exception instead of silently producing a cutoff.

4.3 RDF-to-cutoff conversion

The public cutoff type is defined in `mdstats/analysis/cutoffs.py`:

```

cutoff = PairCutoff.from_rdf_minimum(
    rdf_result,
    minimum_options={...},
)

```

The resulting PairCutoff records:

- canonical species pair;
- minimum radius;
- feature confidence;
- smoothing and search settings;
- source frame and selection provenance.

Each RDF group must contain one unique species for this conversion. The RDF function itself supports broader species selections, but a single PairCutoff cannot represent a mixed-species pair. This object is the recommended input to coordination and bond-angle analysis.

4.4 Implemented RDFResult helpers

```
result.bin_width
result.smoothed(sigma=0.05)
result.first_peak(...)
result.first_minimum(...)
result.coordination_at(cutoff, interpolate=True)
result.first_shell_coordination(
    cutoff=None,
    return_feature=False,
    **minimum_options,
)
result.to_dataframe() # requires pandas
result.save_npz(path)
```

save_npz() is one-way in this module; no RDFResult.load_npz() method is implemented. first_shell_coordination() returns an automatically detected minimum unless a manual cutoff is supplied. With return_feature=True, it also returns the corresponding RDFFeature.

5. Public API

```
def compute_pair_rdf(
    collection: AtomisticFrameCollection,
    species_a: SpeciesSelection = None,
    species_b: SpeciesSelection = None,
    *,
    r_max: float | None = None,
    n_bins: int = 300,
    frame_start: int | None = None,
    frame_stop: int | None = None,
    frame_step: int = 1,
    atom_indices_a: ArrayLike | None = None,
    atom_indices_b: ArrayLike | None = None,
    block_size: int = 256,
    neighbor_search_options: NeighborSearchOptions | None = None,
) -> RDFResult:
    ...
```

5.1 Inputs

collection

Requires atom identities, cells, PBC flags, and positions. The function uses wrapped positions derived from the collection.

Species and explicit selections

At least one of species_a or atom_indices_a must identify group *A*, and likewise for group *B*. If both forms are supplied, explicit indices must match the requested species.

Selections must be identical or disjoint. Empty, duplicate, out-of-range, or partially overlapping explicit selections are rejected.

r_max

Maximum histogram radius in angstrom. If None, the function chooses the exact global unique minimum-image radius over selected frames.

The requested value must satisfy

$$0 < r_{\max} \leq r_{\text{safe}},$$

where

$$r_{\text{safe}} = \frac{1}{2} \min_t \min_{\mathbf{n} \neq 0} \|\mathbf{n}H_t\|,$$

with integer coefficients restricted to periodic axes. The shared neighbor layer evaluates this shortest periodic translation using a validated Minkowski-reduced basis. The older perpendicular-face-height check was conservative for skewed primitive cells and is no longer the RDF limit.

n_bins

Number of equal-width radial bins. Constraint:

`n_bins >= 2`

Frame selection

Standard Python slice semantics. At least one frame must be selected and `frame_step > 0`.

block_size

Center-block size used by the dense oracle. It controls temporary pair-geometry memory, not RDF normalization.

neighbor_search_options

Optional NeighborSearchOptions. None uses the production default: conservative automatic dense/cell-list selection, deformation-aware Verlet reuse only for eligible time-ordered trajectories, and stateless execution for single-frame selections and independent ensembles. Explicit `cache_mode="verlet"` is an expert override. Backend and cache choices change only execution and provenance; RDF counts and normalization are invariant.

6. High-level algorithm

```
resolve frames and atom groups
|
validate r_max through shared neighbor layer
|
construct radial bins and shell volumes
|
for each frame:
    build shared pair geometry up to r_max
    histogram returned distances
    normalize with the instantaneous volume
    accumulate cumulative coordination counts
|
average frame-normalized RDF curves
|
construct RDFResult
```

Pseudo-code:

```
for frame in frame_indices:
    pairs = build_neighbor_list(
        collection,
        frame_index=frame,
        center_indices=A,
        candidate_neighbor_indices=B,
        cutoff=r_max,
        pair_counting=pair_mode,
    )

    frame_counts = histogram(pairs.distances, bin_edges)
    g_frame = normalize(frame_counts, V_frame, N_A, N_B)

    total_counts += frame_counts
    g_sum += g_frame
    coordination_sum += cumulative_neighbor_count(frame_counts)

g_r = g_sum / n_frames
coordination_number = coordination_sum / n_frames
```

The RDF does not recompute minimum-image geometry independently of the coordination and angle modules.

7. Smoothing and feature detection

7.1 Gaussian smoothing

For detection and plotting only,

$$\tilde{g}(r) = G_\sigma * g(r).$$

The user supplies σ in angstrom; the implementation converts it to bin units.

Raw `g_r`, raw counts, and cumulative coordination remain unchanged.

7.2 First peak

`RDFResult.first_peak()` finds the first significant maximum in a user-defined search range using prominence and width criteria.

7.3 First minimum

`RDFResult.first_minimum()` first identifies the first structural peak, then searches for the subsequent significant minimum. The method may repeat the detection under nearby smoothing widths and report

$$\sigma_{r_{\min}}$$

as a stability diagnostic.

A missing, shallow, or smoothing-sensitive minimum raises `AmbiguousFirstMinimumError` unless the user explicitly loosens the detection criteria.

8. Variable cells and frame ensembles

The RDF is valid for variable cells because each frame is normalized by its own volume. A one-frame result is a static pair distribution without temporal averaging.

For a statistical ensemble, frame-weighted averaging gives each structure equal total weight. If structures represent a deliberately nonuniform sampling measure, the resulting RDF reflects that sampling measure. Weighted frames are not part of the first API.

9. Edge cases and warnings

The function rejects:

- invalid or singular cells;
- nonfinite positions;
- unsafe radial ranges;
- empty selections;
- partially overlapping selections;
- invalid bins, frames, or block sizes.

The function records diagnostic messages in `result.metadata["warnings"]` when:

- very few frames or pairs contribute;
- $g(r)$ does not approach a sensible large- r baseline because the cell or selected range is too small;
- finite-size effects dominate the requested range;
- the selected radius approaches the minimum-image limit;
- the collection is not periodic in all three directions.

`compute_pair_rdf()` does not emit Python warning objects for these sampling diagnostics. Confidence for a later RDF-derived cutoff is stored in the `RDFFeature` and `PairCutoff` provenance; consuming modules decide whether to emit a warning.

A first RDF minimum is not always a chemically meaningful bond cutoff. Broad, overlapping, or multi-modal shells require user judgment.

10. Shared-neighbor refactor requirements

The implemented RDF module delegates the following numerical geometry to `_neighbors.py`:

- minimum-image vector and distance calculation;
- conservative safe-radius calculation;
- blocked center-candidate pair enumeration;
- directed versus unordered-identical pair retention.

`rdf.py` retains observable-specific logic that cannot be moved into the geometry kernel: identical/disjoint selection classification for RDF normalization, volume calculation, lightweight frame validation and PBC diagnostics, selection labels, histogram normalization, and feature detection.

The refactor must preserve the numerical RDF and cumulative coordination results within floating-point tolerance.

11. Testing requirements

Tests must cover:

1. Ideal-gas-like normalization on synthetic random structures.
2. Identical and disjoint pair selections.
3. Exact shell-volume normalization.
4. Variable-cell framewise normalization.
5. Orthogonal and triclinic PBC.
6. Agreement with shared-neighbor raw pair counts.
7. Cumulative coordination agreement with integer coordination means at the same cutoff.
8. Stable first peak and first minimum detection.
9. Exact shortest-translation cutoff acceptance for skewed LTA primitive cells.
10. Ambiguous-minimum rejection above the exact unique-image radius.
11. `PairCutoff.from_rdf_minimum()` provenance.
12. Single-frame, ensemble, and trajectory inputs.
13. Strict exclusion of pairs exactly at `r_max`.

12. Deliberate non-goals

This module does not implement:

- bond-angle or full three-body distributions;
- partial structure-factor scattering weights;
- weighted frames;
- dynamic bond persistence;
- automatic chemical interpretation of RDF minima;
- nearest-neighbor or Voronoi definitions.

13. Example workflow

```
rdf = compute_pair_rdf(
    collection,
    species_a="Na",
    species_b="O",
    r_max=6.0,
    n_bins=600,
)

minimum = rdf.first_minimum(
    smoothing_sigma=0.05,
    search_start=1.5,
    search_max=4.5,
)
```

```

na_o_cutoff = PairCutoff.from_rdf_minimum(
    rdf,
    minimum_options={
        "smoothing_sigma": 0.05,
        "search_start": 1.5,
        "search_max": 4.5,
    },
)

```

The same `na_o_cutoff` can then be inserted into a `PairCutoffRegistry` and reused by coordination and O-Na-O angle analysis. Registry mappings and nested cutoff provenance are read-only after construction.

14. Reproduction checklist

An independent implementation reproduces this module when it:

1. uses shared minimum-image pair geometry;
2. counts disjoint pairs as directed and identical pairs as unordered;
3. normalizes every frame with its own volume;
4. uses exact shell volumes;
5. derives cumulative coordination from raw pair counts;
6. never derives authoritative coordination from a smoothed RDF;
7. preserves feature-detection diagnostics;
8. produces reusable `PairCutoff` provenance from a selected first minimum;
9. accepts `r_max` up to half the exact shortest periodic translation and rejects larger single-image requests.

15. Stage S4 execution policy

One analysis-local neighbor executor is shared across all selected RDF frames. The result stores deterministic execution diagnostics in `metadata["neighbor_search"]`. The normative policy, fallback, cache, and provenance contract is `docs/specs/analysis/neighbor_search_spec.md`.

16. Minimum-image cutoff provenance

The RDF module delegates shortest-vector evaluation to the shared neighbor layer specified in `_neighbors_spec.md`. That layer uses ASE's Minkowski-reduction implementation [1], based on the low-dimensional reduction algorithm of Nguyen and Stehle [2].

17. References

1. Larsen, A. H., et al. (2017). *The Atomic Simulation Environment - A Python Library for Working with Atoms*. Journal of Physics: Condensed Matter, 29(27), 273002. DOI: 10.1088/1361-648X/aa680e.
2. Nguyen, P. Q., and Stehle, D. (2009). *Low-Dimensional Lattice Basis Reduction Revisited*. ACM Transactions on Algorithms, 5(4), Article 46. DOI: 10.1145/1597036.1597050.